FormulaSync: A Coordination of Shared State and Formulas in Collaborative Spreadsheets

Korhan Deniz AKIN

Contents

1 General Information **3**

2 Problem Definition **3**

3 Related Work **3**
3.1 Semantics for CRDT-based Collaborative Spreadsheets 3
3.2 AegisSheet: A Compositional CRDT for Collaborative Spreadsheets 3
3.3 Collabs Spreadsheet Prototype 4
3.4 Bartosz Sypytkowski's 2D Table CRDT 4
3.5 Concordant CRDT Spreadsheet 4
3.6 FortuneSheet 4
3.7 TinyCld Calc 4
3.8 Framacalc 4

4 Representing Spreadsheet Data in Yjs **4**
4.1 Overview 4
4.2 YColumns & YRows (SpreadSheet.tsx) 4
4.3 YMap (Cell.tsx) 5
4.4 YColKeep & YRowKeep (SpreadSheet.tsx) 5

5 Formula Support **5**
5.1 Information 5
5.2 Implementation Details 6
5.2.1 Formula Parser 6
5.2.2 Spreadsheet Store 6
5.2.3 SpreadSheet Component 7
5.2.4 Cell Component 7
5.3 Circular Reference Handling 7
5.4 Benchmarking 8

6 CSV Support **8**

1 General Information

In this research, the aim is to build a collaborative spreadsheet that supports formula parsing like Microsoft Excel. React.js and TypeScript are used to build the application. Typescript is chosen instead of Javascript because it provides strict typing. Synchronization across multiple clients was achieved by Yjs, which is a JavaScript framework that uses Conflict-free Replicated Data Types (CRDTs) with an abstraction of shared data [1]. Zustand is used for global React state management. Real-time updates are transmitted to the connected peers using a WebSocket layer which is implemented by y-websocket.

2 Problem Definition

3 Related Work

3.1 Semantics for CRDT-based Collaborative Spreadsheets

Our research builds upon the work of Yanakieva et al. (2023). In their work, they compared several data models on implementing collaborative spreadsheets and proposed an implementation which supports conflict resolution for offline operations using a combination of existing replicated data types provided by Yjs. They proposed a new data model called "Fixed Elements (Remove-keep)" to prevent data loss when a user modifies a cell and, at the same time, another user deletes the corresponding column or row. They did this by using two y-arrays (to keep track of the orders of cell ids), a y-map (to associate cell ids to their contents) and two "keep" y-maps for columns and rows. The idea is to store the "keep" ids in these maps when there is a "positive update" on a cell. If there is a removal opeartion concurrent to the positive update, the "keep" ids are used to undo the removal operation. Y.UndoManager is used to implement the undo operation. This implementation is used in our research as well. Update-wins CRDT for cell contnet is also implemented in our research which was not present in the original work.

3.2 AegisSheet: A Compositional CRDT for Collaborative Spreadsheets

Another relevant work is AegisSheet, which is a compositional CRDT for collaborative spreadsheets. It was implemented by Pfeil et al. (2026).

3.3 Collabs Spreadsheet Prototype

3.4 Bartosz Sypytkowski's 2D Table CRDT

3.5 Concordant CRDT Spreadsheet

3.6 FortuneSheet

3.7 TinyCld Calc

3.8 Framacalc

4 Representing Spreadsheet Data in Yjs

4.1 Overview

Representing spreadsheet in a collaborative environment was quite straightforward. Yjs framework was very useful because it abstracts all the network broadcasting messages, CRDT implementations, merging updates with CRDTs, and even WebSocket connections. Therefore, none of these aspects are needed to be reimplemented. However, using this framework with React's state management was the hardest part because Yjs only solves synchronization between other clients not how the client's screen is updated and how does the data is displayed. In relevant sections this will be explained further.

The implementation uses 2 arrays and a map. These arrays store cell IDs and are crucial to display them on the table with respect to the order. The map is used to store the content of the cells. This means there is no global grid, which might cause bottlenecks during adding/removing columns/rows so, instead of storing the entire table as a matrix, the sheet is stored into separate structures.

4.2 YColumns & YRows (SpreadSheet.tsx)

These are CRDT-based arrays that are responsible for ordering cells in the spreadsheet. Using a map structure is not enough to ensure ordering, because maps are unordered.

They appear in the SpreadSheet.tsx component. In this component, YColumns and YRows are used to transfer cell IDs to the Spreadsheet's React State when this component is mounted. Later, 'columns' and 'rows' states are used in the table to display the content. They show how the spreadsheet synchronizes the Yjs shared column data with the local React state. Because Yjs data structures live outside of the React state, changes in Yjs do not trigger a re-render in the user interface (as mentioned before). An observer function inside 'useEffect' bridges this gap. When a user adds or removes a column, the observer function receives an event:

- Insertions: When new column IDs are inserted, their target index is found using 'findYArrayElement' function. The new column is inserted into the

local 'columns' array.

- Deletions: Deleted column IDs are collected from into a data structure. Then the local React state is then updated by filtering out these deleted columns.

4.3 YMap (Cell.tsx)

To keep values of each cell, YMap structure was used. It's a CRDT-based key-value (Yjs provided abstraction for CRDTs as mentioned) pair where ids are defined as:
`let cellId: string = ‘${col.id},${row.id}‘;`
and contents are just texts. To use the YMap effectively, for every Cell on the spreadsheet, an observer is defined. This observer is responsible for keeping track of the remote updates of the Cell's own content. This comes with a performance cost, but the benefits are not negligible. This was the most convenient approach so far rather than using a global grid.

First, with this approach, storing Cell components in a global grid was not needed. If there was a global grid, adding or removing rows/columns would be costly because of shifting all the remaining elements.

This observer also has an implementation for Update-Wins CRDT. The idea behind it is to store updates that are concurrent to each other (e.g., offline clients performing updates) and merge them by concatenation. That's the reason why YMap entries have arrays as their values. Later appendConcurrentUpdates() is called to do string concatenation (merging concurrent updates).

4.4 YColKeep & YRowKeep (SpreadSheet.tsx)

With the standard Yjs, data was lost when a user modified a cell and, at the same time, another user deleted the corresponding column or row. SpreadSheet addresses such data loss by using "keep" arrays for columns and rows [5]. When there is a "positive update" on a cell: "keep id"
`let keepId: RemoveKeepOperationId = ‘c${YDoc.clientID as number}.${1 as number}‘;`
is stored in those arrays. Clients monitor the changes of arrays with observers, and if they are non-empty, "undo" operations are performed equal to the number of elements in the array structures.

5 Formula Support

5.1 Information

Supporting formulas in a collaborative environment would be interesting for several reasons:

1. The implementation itself.

2. Retrieving spreadsheet data for formula calculation. What is the feasible way of doing this without ruining the spreadsheet if there are offline users too?

3. Formula recalculation for every update and its probable bottlenecks. For example, let's say a result is computed for a formula using 1000 cells and a user updates one cell value by incrementing it by one. In this scenario, should the formula be calculated from the very beginning, or is it possible to support this functionality more efficiently? While the current implementation does so by recalculating the formula, hard coding improvements by addressing supported formulas is possible.

4. What happens when a new row or column is added in between formula reference cells? (To be implemented)

5. Addressing probable issues such as circular referencing during calculation and recalculation.

5.2 Implementation Details

In this section, relevant parts of functions/components for formula calculation will be explained.

5.2.1 Formula Parser

First, the `‘fast-formula-parser‘` [2] library is used for formula functionality. The main implementation for this library is in 'formula.ts'. In this file, there are 2 functions: 'parser()', which comes from the library itself, and 'parserDriver()' which calls the parser() with 'spreadsheetData' obtained in the Spreadsheet component.

This parserDriver() function returns the formula result and returns the IDs of the cells that have been marked (or registered) while computing the formula result.

5.2.2 Spreadsheet Store

To retrieve the store globally, Zustand, which is a small, fast, and scalable state management solution [4], is used. It is like Redux but was easier to implement.

Normally, the direction for the data transfer is from the parent to the child. However, it would not have been feasible to transfer all spreadsheet data from the Spreadsheet component to every Cell. The second solution requires spreadsheet data*num of total cells. With Zustand, there is a single spreadsheet used for formula calculations.

'spreadSheetData' is the name of the object that contains all values in each cell and is a matrix of strings obtained from 'SpreadSheetStore.ts': 'returnSpreadSheet()'. As mentioned in section 3.1 this function returns all spreadsheet data (optimization might be possible).

5.2.3 SpreadSheet Component

The SpreadSheet component provides a table by implementing the PrimeReact library's DataTable structure with the Yjs framework. While the table's rows and columns are tracked via Yjs arrays (yRows, yColumns), observers ensure that all additions and deletions are reflected in the local React state. When a new row or column is created, the corresponding cells are automatically created in the shared map, and deletion operations trigger both map cleanup and undo mechanisms. This component also consists of other components, which are headers (ColumnHeader), cells (Cell), and buttons.

handleFormula()

5.2.4 Cell Component

handleFocusOut() Formula cells register candidate cells in the 'markedCells[]' array when the formula is defined to compute the result. Similarly, candidate cells store IDs of the formula cells in the 'formulaReferenceCells[]' array. This is an array because there might be multiple formula cells registering the same cell. These 2 arrays are kept in every Cell component and are defined in 'CellType'. Arrays are optional, denoted with "?", so it is not a problem if they are undefined.

After these operations, the 'handleFocusOut()' function in the Cell component is responsible for the formula recalculation as shown in Figure **??**.

5.3 Circular Reference Handling

A circular reference occurs when a formula refers directly or indirectly back to its own cell value, creating an endless calculation loop. For example, if cell A1 contains the formula =A1+B1, then A1 references itself [3]. Actually, it is not an issue when defining the formula (the cell is not yet related to the corresponding formula). However, when there are updates to cells marked by formulas, a circular reference becomes apparent. Meaningly, when there is formula recalculation (as explained in section 0).

The implementation does not produce an endless loop because formula recalculation does not use observers. Rather, it is being done via the 'handleFocusOut()' function in Cell components. Therefore, a formula is calculated via 'handleFocusOut()' calls. Even though there are no calculation infinite loops, when a user wants to increase a cell value by one (if the cell is marked and the formula is SUM), as in a circular reference case, formula cells register themselves, the existing result from the previous step, so it does not increase by one rather formula's previous result + 1. This produces a result that the user does not intend.

To solve this, Microsoft Excel's approach is used. When it detects a circular reference, the formula value is set to 0. In my implementation, I have a very

basic approach that can be seen in Figure **??**. This approach might be improved in terms of time complexity via Depth-First Search.

5.4 Benchmarking

6 CSV Support

References

[1] Kevin Jahns. *Yjs: Shared Data Types for Building Collaborative Software*. https://github.com/yjs/yjs. Accessed: 2026-08-23. 2026.

[2] Lumen Lyu. *fast-formula-parser*. https://www.npmjs.com/package/fast-formula-parser. Accessed: 2026-07-24. 2020.

[3] Macabacus. *Circular Reference in Excel: How to Find and Fix Errors*. https://macabacus.com/blog/conquering-circular-references-in-excel. Accessed: 2026-07-23. 2026.

[4] Poimandres. *Introduction - Zustand Documentation*. https://zustand.docs.pmnd.rs/learn/getting-started/introduction. Accessed: 2026-07-25. 2026.

[5] Elena Yanakieva, Philipp Bird, and Annette Bieniusa. "A Study of Semantics for CRDT-Based Collaborative Spreadsheets". In: *Proceedings of the 10th Workshop on Principles and Practice of Consistency for Distributed Data*. PaPoC '23. New York, NY, USA: Association for Computing Machinery, 2023, pp. 37–43. DOI: 10.1145/3578358.3591324.